

AI CALLER AGENT

Testing Guide

What we test, why, and who owns it

Version 1.1

Cross-referenced throughout the Architecture Specification and Team Execution Plan

Every category here maps to §34 of the Architecture Specification and §K of the Team Execution Plan

Table of Contents

1. Why This Many Kinds of Testing?	3
2. Unit Testing	3
3. Integration Testing	4
4. Voice Testing	4
5. RAG Testing	5
6. Security Testing	5
7. Tenant Isolation Testing	6
8. Performance Testing	6
9. Load Testing	7
10. Regression Testing	7
11. User Acceptance Testing (UAT)	8

1. Why This Many Kinds of Testing?

CONCEPT · The testing pyramid

WHAT IT IS

Different kinds of tests catch different kinds of bugs, at different costs to run -- the pyramid shape reflects running lots of cheap, fast, narrow tests constantly, and fewer expensive, slow, broad tests less often.

REAL-WORLD ANALOGY

A car factory doesn't crash-test every single car (too slow, too expensive, destroys the car) -- but it does electrically test every single unit off the line. It crash-tests a sample, periodically, because that test catches a different category of problem that a wiring check never could.

WHY STAYKARO USES IT

A unit test can tell you a function returns the wrong value in under a second. It cannot tell you whether a real caller speaking Telugu over a bad mobile connection gets a good experience -- that needs a voice test, which takes minutes, not milliseconds, and can't run on every keystroke.

TECHNICAL IMPLEMENTATION

Ten categories below, each with a place in the pyramid, a trigger (when it runs), and an owner.



Figure 1.1 -- The testing pyramid for this project. Lower layers run more often; higher layers run less often but catch what lower layers structurally cannot.

2. Unit Testing

Why it exists: Unit tests check one function or class in isolation, with everything around it faked or stubbed out.

What bugs it catches: Logic errors in a single piece of code -- an off-by-one error, a wrong conditional, a miscalculated value -- caught in under a second, long before the code ever touches a real database or a real call.

When: On every commit, automatically, in CI (Developer Handbook §11) -- before a human reviewer ever looks at the PR.

Who's responsible: The author of the code, as part of writing it (Developer Handbook §13: "a PR without a test does not get merged")

Example test cases

- usage_records calculation: given known telephony/STT/LLM/TTS inputs, the computed provider_cost_inr matches a hand-calculated expected value
- TenantNotFoundError is raised (not a silent None) when a tenant_id doesn't exist
- JWT verification rejects an expired token even with a valid signature

Pass / fail criteria

- 100% of new business logic has at least one unit test; a test that passes whether or not the fix is applied does not count (Code Review Checklist, Developer Handbook §11)

3. Integration Testing

Why it exists: Integration tests check that two or more real components -- the API and a real (test) PostgreSQL, or the RAG service and a real (test) pgvector index -- work correctly together.

What bugs it catches: Bugs that only appear at the boundary between components: a schema mismatch, a wrong SQL query that still "type-checks," a webhook handler that works against a mocked signature but not a real one.

When: On every PR touching more than one component, in CI, against a throwaway test database (Database Design §9)

Who's responsible: The author, reviewed by whoever owns the other side of the boundary being tested (Execution Plan §K)

Example test cases

- A knowledge document upload flows all the way through extraction, chunking, and embedding into real pgvector rows
- A Razorpay webhook payload (test fixture) correctly updates subscription status in a real test database
- A call-turn event posted to the internal API produces a correctly persisted call_turns row

Pass / fail criteria

- The test exercises real components, not mocks, for at least the boundary being tested; it fails if either side changes its contract without the other being updated

4. Voice Testing

Why it exists: Voice tests place real or recorded-audio calls through the actual pipeline and evaluate the real, spoken experience -- not just that code compiles.

What bugs it catches: Problems invisible to any other test category: bad endpoint tuning that cuts callers off mid-sentence, an accent or code-mixed phrase the STT mishandles, a TTS voice that sounds wrong for a given language.

When: Every PR touching the voice pipeline (smaller scripted set); the full matrix before each Phase 5/8 exit (Architecture Spec §34)

Who's responsible: Shivashree, with Nishkala/Nihal sign-off on the full multilingual matrix (Execution Plan §K)

Example test cases

- English, Hindi, Telugu, Hinglish, and Telugu-English scripted conversations
- A call with a mid-sentence pause, verifying the AI doesn't cut in prematurely
- A call over deliberately degraded/simulated poor network conditions
- Caller interrupts (barge-in) mid-response and the AI stops within acceptable latency

Pass / fail criteria

- The full language matrix (Architecture Spec §16 for retrieval, §34 for voice) passes with no systematic quality gap between English and Hindi/Telugu

5. RAG Testing

Why it exists: RAG evaluation checks retrieval and answer quality against a known question/answer benchmark set, not just "does the code run without an error."

What bugs it catches: The specific, hard-to-notice failure mode where the agent answers fluently and confidently from the wrong -- or no -- retrieved context, which looks identical to a correct answer unless you know the real one.

When: On any change to ingestion, retrieval, or the embedding model; the full per-client benchmark before a pilot client goes live

Who's responsible: Nishkala, with the benchmark set built per pilot client (Architecture Spec §34)

Example test cases

- A question answerable only from an uploaded document returns the correct, grounded answer
- A question with no answer in the knowledge base results in a safe "I don't know," not a fabricated answer
- A Telugu question against an English-language knowledge base still retrieves the right chunk (Database Design §16 matrix)

Pass / fail criteria

- Retrieval relevance and factual correctness meet the per-client benchmark threshold; the unsupported-answer (fabrication) rate is effectively zero

6. Security Testing

Why it exists: Security testing deliberately tries to break the system's protections -- auth, input validation, secrets handling -- the way an attacker would, not the way a normal user would.

What bugs it catches: Broken access control, injection vulnerabilities, secrets leaking into logs or responses, webhook endpoints that trust unverified input.

When: Every PR touching auth, RBAC, or an externally-reachable endpoint, in CI; a fuller pass before each production hardening milestone (Phase 10)

Who's responsible: Whoever reviews the PR (Developer Handbook §12 Security Checklist), with a dedicated pass owned by Nishkala at Phase 10

Example test cases

- An expired or tampered JWT is rejected on every protected endpoint, not just some
- A Razorpay webhook with an invalid signature is rejected before its body is processed at all
- No API response, error message, or log line leaks a secret, password hash, or another tenant's data

Pass / fail criteria

- Every item on the Security Checklist (Developer Handbook §12) passes; no high-severity finding open at launch

7. Tenant Isolation Testing

Why it exists: An automated suite that logs in as one tenant and deliberately attempts to read or modify another tenant's data through every surface the platform exposes.

What bugs it catches: Cross-tenant data leakage -- the single most consequential bug category in the entire system, since it's a trust and legal problem, not just a functional one.

When: In CI on every PR touching tenant-scoped code; the full suite is the hard gate for Checkpoint 4 (Execution Plan §I) and re-run before every production release afterward

Who's responsible: Nishkala implements; Shivashree or Nihal reviews (Execution Plan §K -- this is the one test category with a mandatory non-author reviewer even for the tests themselves)

Example test cases

- Client A's JWT used against Client B's `call_id`, `document_id`, and `agent_id` on every relevant endpoint -- all must fail
- A RAG query from Client A's call never returns a chunk tagged with Client B's `tenant_id`, under concurrent load from both tenants simultaneously
- An export or report endpoint scoped to "my tenant" cannot be coerced into returning another tenant's rows via a crafted parameter

Pass / fail criteria

- Zero successful cross-tenant access across every attempted surface, under both sequential and concurrent test conditions -- this suite has no acceptable failure rate

8. Performance Testing

Why it exists: Performance testing measures actual latency, per pipeline stage, under realistic conditions -- not just "does it eventually respond."

What bugs it catches: A slow RAG query, a provider integration adding unexpected overhead, an endpoint-detection setting that's technically correct but subjectively sluggish.

When: On every change to the voice pipeline or RAG retrieval path; full stage-by-stage review at Phase 2 and Phase 10

Who's responsible: Shivashree (voice pipeline stages), Nishkala (RAG/database stages)

Example test cases

- Per-stage latency (endpoint detection, STT, RAG, LLM TTFT, TTS TTFA) measured and logged for a sample of real calls, per Architecture Spec §20

- Latency measured specifically during LLM fallback (Groq → OpenAI), not just the primary path
- Latency measured over a simulated real mobile network, not only the office network

Pass / fail criteria

- End-of-speech to first-audible-response approaches the <1.5s target under healthy conditions, measured -- not assumed from summing best-case stage numbers

9. Load Testing

Why it exists: Load testing drives many simultaneous synthetic calls through the system to find the point where it degrades, and to prove a concurrency number rather than guess one.

What bugs it catches: Bottlenecks invisible at low volume: a connection pool that's too small, a provider rate limit that's fine for 5 calls but not 50, a database query that's fast alone but slow under contention.

When: At Phase 10 before launch; re-run before any claim of increased capacity afterward

Who's responsible: Nihal (infrastructure/harness), Shivashree (voice-side profiling), Nishkala (database-side profiling)

Example test cases

- 10, 25, 50, and 100 concurrent synthetic calls (Architecture Spec §34), with per-stage latency tracked at each level
- Confirming whether Groq's rate limits hold at the target concurrent volume, or whether OpenAI fallback triggers more than expected under load
- Database and Redis latency under concurrent tenant-mixed load, not single-tenant load

Pass / fail criteria

- A documented, measured maximum concurrent-call capacity exists before launch; concurrency capacity is never claimed without this test having actually been run (Architecture Spec §34)

10. Regression Testing

Why it exists: Regression testing re-runs previously-passing tests to confirm a new change hasn't quietly broken something that used to work.

What bugs it catches: The specific failure mode of fixing one thing and silently breaking another -- common in a system with as many interacting parts (voice, RAG, tenancy, billing) as this one.

When: Automatically, as part of every CI run (the full existing test suite, not just new tests) -- and a focused manual pass before each release, covering areas the specific release touched

Who's responsible: Automated for CI; manual regression pass owned by whoever is coordinating the release

Example test cases

- Full existing test suite (all categories above) passes, not just tests for the new change
- A previously-fixed bug (kept as a named regression test) stays fixed
- A change to the billing module doesn't silently break the voice pipeline's tests, and vice versa

Pass / fail criteria

- 100% of the existing automated suite passes before merge; no previously-fixed, regression-tested bug reappears

11. User Acceptance Testing (UAT)

Why it exists: UAT is a real person -- ideally a pilot client, or someone playing that role -- using the finished feature the way an actual user would, evaluating whether it's actually good, not just technically correct.

What bugs it catches: The gap between "passes every automated test" and "a real business owner would actually be happy with this" -- tone of the agent's responses, whether the dashboard is genuinely usable, whether the pricing answer sounds natural rather than robotic.

When: Before each pilot client goes live (Phase 11), and for any client-facing change significant enough to warrant a fresh look

Who's responsible: The pilot client themselves where possible; Staykaro's team role-playing the client otherwise

Example test cases

- A pilot client's own team places real calls to their configured agent and gives direct feedback (Architecture Spec §42, Pilot Success Metrics)
- A client walks through configuring their agent and uploading a document in the dashboard, unassisted, and succeeds
- A non-technical stakeholder reviews a sample of real call transcripts for tone and accuracy, not just correctness

Pass / fail criteria

- Direct, collected (not inferred from silence) client feedback per Architecture Spec §42; no unresolved UAT-raised issue blocking that client's launch